

Advanced Optimization

NEURAL ARCHITECTURE SEARCH WITH REINFORCEMENT LEARNING

René Mourambou Diyema

`rene.mourambou-diyema@universite-paris-saclay.fr`
`garciagarcia293@gmail.com`

Université Paris-Saclay

Contents

1	Introduction	3
2	Methodology and Theoretical Framework	3
2.1	Neural Architecture Search via Reinforcement Learning	3
2.2	Skip Connections via Set-Selection Attention	4
2.3	Recurrent Cell Architecture Generation	5
3	Experimental Setup	6
3.1	Datasets and Preprocessing	6
3.2	Model Configurations and Hyperparameters	6
3.3	Search Space and Training Protocol	7
4	Experimental Results and Performance Analysis	7
4.1	Convolutional Architecture Search on CIFAR-10	7
4.2	Recurrent Cell Search on Penn Treebank Word-Level	8
4.3	Character-Level Transfer on Penn Treebank	9
4.4	Control Experiments	10
5	Critical Analysis and Interpretation of Results	10
5.1	Consistency with the Original Paper	10
5.2	Limitations and Sources of Discrepancy	11
6	Conclusion	12
A	Hyperparameters	13
A.1	CIFAR-10 Configuration	13
A.2	Penn Treebank Configuration	14
B	Results Tables	15
B.1	CIFAR-10 Results	15
B.2	Penn Treebank Word-Level Results	16
B.3	Penn Treebank Character-Level Results	16
B.4	NAS vs Random Search — Reward Comparison	16
C	Pseudocode of Core Algorithms	17
C.1	Neural Architecture Search — Main Loop	17
C.2	REINFORCE Policy Gradient with Baseline	18
C.3	Recurrent Cell Forward Pass	19

1 Introduction

Designing neural network architectures has traditionally been a manual and time-consuming process, requiring significant domain expertise and iterative experimentation. Researchers and practitioners typically rely on intuition, empirical guidelines, and trial-and-error to build models that perform well on specific tasks. While powerful architectures such as AlexNet, VGGNet, and ResNet have demonstrated the importance of architectural decisions on model performance, the process of arriving at these designs remains largely ad hoc and labour-intensive. Neural Architecture Search (NAS), introduced by Zoph & Le (2017), proposes an automated solution to this problem by casting architecture design as a reinforcement learning problem: a recurrent neural network controller generates candidate architectures, trains them on a target task, and uses the resulting validation performance as a reward signal to iteratively improve its search policy. The framework is evaluated on two benchmark tasks — image classification on CIFAR-10 and language modelling on Penn Treebank — where it discovers architectures that rival or surpass the best human-designed models of the time.

This report presents a replication of the NAS framework proposed by Zoph & Le (2017), targeting both the CIFAR-10 convolutional architecture search and the Penn Treebank recurrent cell search, together with the control experiments comparing NAS against random search and the transfer learning experiment on character-level language modelling. Due to significant computational constraints — the original paper uses 800 GPUs for CIFAR-10 and 400 CPUs for PTB, running for several days — this replication operates at a substantially reduced scale. The purpose is therefore not to match the exact numerical results reported in the paper, but to faithfully validate each component of the NAS pipeline and to confirm that the overall framework behaves as described. The remainder of this report is structured as follows: Section 2 provides the theoretical background, Section 3 describes the experimental setup, Section 4 presents the results obtained, Section 5 discusses their coherence with the original paper and identifies key limitations, and Section 6 concludes the report.

2 Methodology and Theoretical Framework

2.1 Neural Architecture Search via Reinforcement Learning

Neural Architecture Search (NAS) is a meta-learning framework in which the architecture of a neural network is treated as a learnable object. The key observation underlying NAS is that the structure and connectivity of a neural network can be encoded as a variable-length string of discrete tokens, and that a recurrent neural network — referred to as the *controller* — can be trained to generate such strings. The controller is implemented as a two-layer LSTM with 35 hidden units per layer, weights initialised uniformly in $[-0.08, 0.08]$, and optimised with the Adam optimizer. At each step, the controller produces a softmax distribution over the possible choices for the current hyperparameter (filter height, filter width, number of filters, stride), samples from it, and feeds the sampled token back as input for the next step, making the policy auto-regressive. The entire sequence of T predictions defines one complete architecture, which is then used to build and train a *child network* on the target task. The validation performance R of the child network is recorded and fed back to the controller as a reward signal, closing the

search loop. The objective is to maximise the expected reward $J(\theta_c) = \mathbb{E}_{P(a_{1:T}; \theta_c)}[R]$, where θ_c are the controller parameters and $a_{1:T}$ is the sampled sequence of architectural decisions. Since R is non-differentiable with respect to θ_c — it requires training an entire child network — the REINFORCE policy gradient rule (Williams, 1992) is used to estimate the gradient as $\nabla_{\theta_c} J(\theta_c) = \sum_{t=1}^T \mathbb{E}_{P(a_{1:T}; \theta_c)}[\nabla_{\theta_c} \log P(a_t | a_{t-1:1}; \theta_c) \cdot R]$, which is approximated in practice over a mini-batch of m sampled architectures as $\nabla_{\theta_c} J(\theta_c) \approx \frac{1}{m} \sum_{k=1}^m \sum_{t=1}^T \nabla_{\theta_c} \log P(a_t | a_{t-1:1}; \theta_c) \cdot (R_k - b)$, where b is an exponential moving average baseline with decay 0.999 subtracted to reduce the variance of the estimator without introducing bias, and R_k is the validation accuracy of the k -th child network. To accelerate training, the paper employs an asynchronous distributed parameter server scheme with $S = 20$ shards, $K = 100$ controller replicas, and $m = 8$ child networks trained in parallel per replica, for a total of 800 concurrent GPU workers.

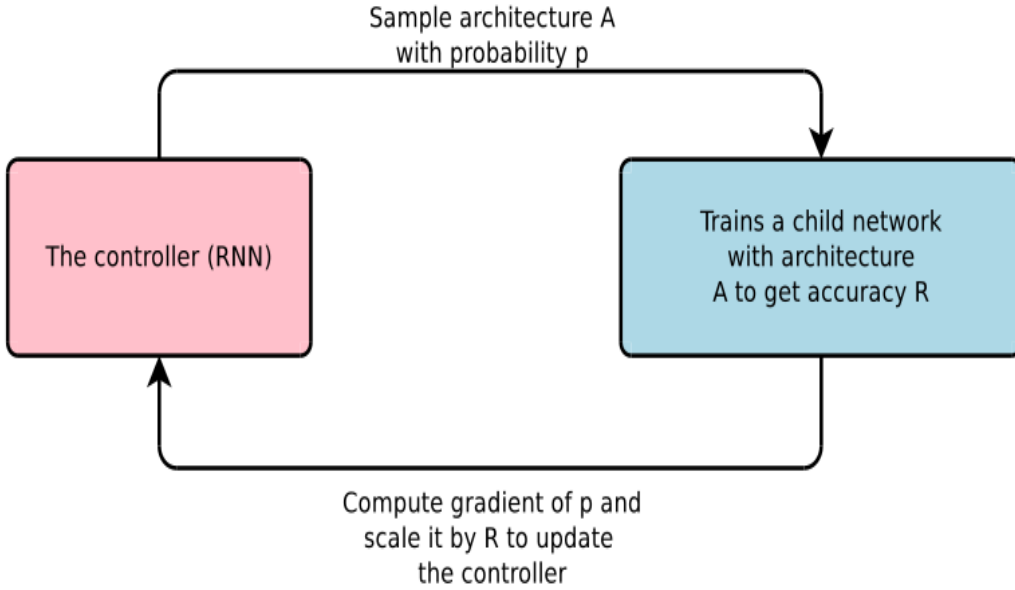


Figure 1: Neural Architecture Search framework. The controller RNN generates architecture descriptions, child networks are trained and evaluated, and the validation accuracy is returned as a reward to update the controller via policy gradient.

2.2 Skip Connections via Set-Selection Attention

The basic NAS search space generates feed-forward chains of convolutional layers, which excludes the richer connectivity patterns found in modern architectures such as ResNet and GoogleNet. To address this, Zoph & Le (2017) introduce a set-selection attention mechanism that allows the controller to propose skip connections between non-adjacent layers. At layer i , the controller adds an anchor point with $i - 1$ content-based sigmoid outputs, one for each previous layer $j \in \{0, \dots, i - 1\}$. The probability that layer j is used as an input to layer i is computed as $P(\text{layer } j \rightarrow \text{layer } i) = \sigma(v^\top \tanh(W_{\text{prev}} h_j + W_{\text{curr}} h_i))$, where h_j and h_i are the hidden states of the controller at the anchor points for layers j and i respectively, and W_{prev} , W_{curr} , v are trainable parameters shared across all layers. Binary skip decisions are sampled from these sigmoid probabilities, and since they are defined by probability distributions, the REINFORCE update applies without modification. When a layer receives multiple inputs, all tensors are concatenated along the channel dimension; if their spatial dimensions differ, smaller tensors are zero-padded to match the largest.

Three boundary rules handle degenerate cases: if a layer receives no selected input, the original image is used as its input; at the final layer, all outputs not consumed by any downstream layer are concatenated before the classifier; and incompatible spatial sizes are resolved by zero-padding.

2.3 Recurrent Cell Architecture Generation

For the Penn Treebank language modelling task, the controller is modified to generate recurrent cell architectures rather than convolutional ones. The computation of a recurrent cell can be generalised as a binary tree that takes x_t and h_{t-1} as inputs and produces h_t as output, where each node is labelled with a combination method in $\{\text{add}, \text{elem_mult}\}$ and an activation function in $\{\text{identity}, \text{tanh}, \sigma, \text{ReLU}\}$. Leaf nodes compute a linear transformation of the inputs, $a_i = \text{act}_i(W_{x,i} x_t + W_{h,i} h_{t-1})$, while internal nodes merge their two children using the specified combination and activation. To incorporate LSTM-style cell state variables, the controller additionally predicts two indices specifying where c_{t-1} is injected into the tree and from which node c_t is read. With a base number of 8 leaf nodes, the total number of nodes in the tree is $2 \times 8 - 1 = 15$, and the resulting search space contains approximately 6×10^{16} distinct cell architectures. The controller visits each node sequentially and predicts its combination method and activation function as two independent softmax classifications, followed by two additional softmax decisions for the cell state injection points, making the total number of controller decisions per cell equal to $2 \times 15 + 2 = 32$ predictions.

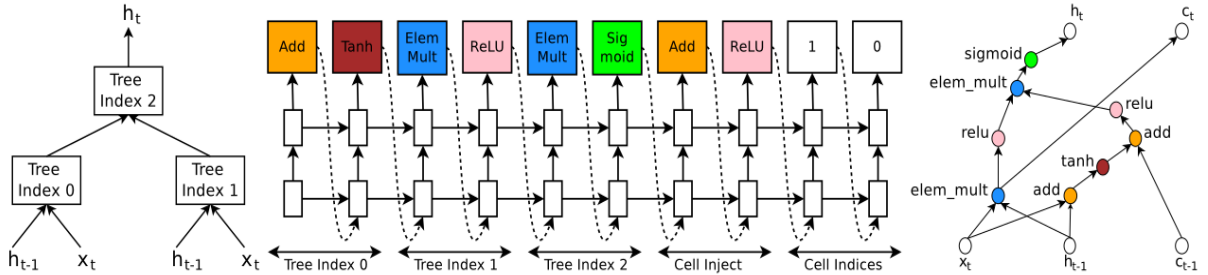


Figure 2: Recurrent cell construction from a tree with two leaf nodes . The controller predicts a combination method and activation function for each node, plus the cell state injection indices.

3 Experimental Setup

3.1 Datasets and Preprocessing

The CIFAR-10 dataset consists of 60,000 colour images of size 32×32 pixels across 10 classes. Following the exact protocol of the paper, the 50,000 training images are split into 45,000 training samples and 5,000 validation samples, with the 10,000 test images held out for final evaluation only. Three preprocessing steps are applied in sequence: global ZCA whitening computed on the full training set via singular value decomposition of the empirical covariance matrix, random upsampling of each image to 40×40 followed by a random crop back to 32×32 , and random horizontal flipping. No data augmentation is applied at test time beyond ZCA whitening. The Penn Treebank dataset, preprocessed following Mikolov et al. (2010), is used for both word-level and character-level language modelling. In its word-level form, it consists of approximately 900K training tokens, 70K validation tokens, and 80K test tokens, with a fixed vocabulary of 10,000 words and an `<eos>` token appended at the end of each line; performance is measured in perplexity (PPL). For the character-level transfer experiment, the same raw text files are read character by character with a vocabulary of all unique characters present in the training file, and performance is measured in bits per character (BPC). Crucially, both the word-level and character-level tasks share the same three data files (`ptb.train.txt`, `ptb.valid.txt`, `ptb.test.txt`), with the only difference being the tokenisation granularity applied at loading time. All data files are downloaded automatically at runtime from their canonical sources and stored in the `./data/` directory of the project.

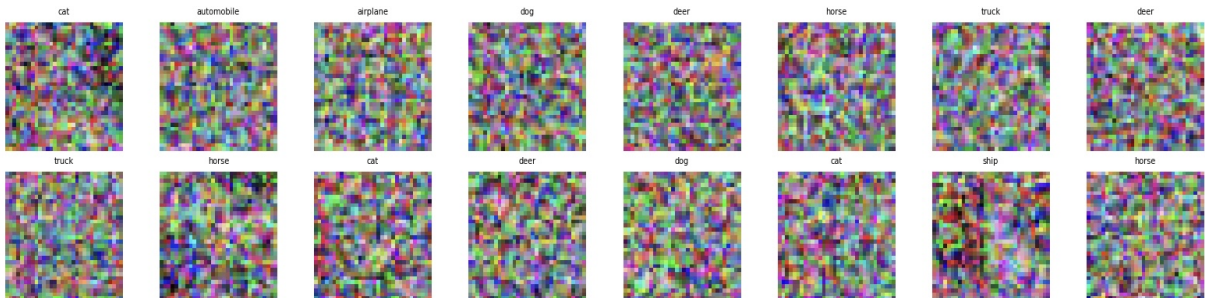


Figure 3: Sample batch of CIFAR-10 images after ZCA whitening preprocessing. The whitening operation decorrelates pixel values and normalises their variance, which is the exact preprocessing used in the original paper.

3.2 Model Configurations and Hyperparameters

The controller is a two-layer LSTM with 35 hidden units per layer, optimised with Adam at a learning rate of 0.0006 for CIFAR-10 and 0.0005 for PTB, with all weights initialised uniformly in $[-0.08, 0.08]$. The reward signal for CIFAR-10 is defined as $R = \max(\text{val_acc}[-5 :])^3$, i.e., the cube of the maximum validation accuracy over the last 5 epochs of child training, while for PTB the reward is $R = -\text{val_perplexity}$, which the controller maximises by minimising perplexity. The exponential moving average baseline has a decay rate of 0.999 in both cases. Child convolutional networks for CIFAR-10 are trained for 50 epochs (5 in our demo) using Nesterov SGD with learning rate 0.1, momentum 0.9, and weight decay 10^{-4} , with the learning rate decayed at 50% and 75% of training; each architecture consists of convolutional layers with batch normalisation

and ReLU activations, and the search space allows filter heights and widths in $\{1, 3, 5, 7\}$, number of filters in $\{24, 36, 48, 64\}$, and stride fixed at 1. Child RNN models for PTB are trained for 35 epochs (3 in our demo) using SGD, with two layers and hidden size adjusted to approximately match the parameter count of the medium LSTM baseline ($\sim 10\text{M}$ parameters in the paper, 200 units in our replication), and regularisation via embedding dropout and recurrent dropout following Zaremba et al. (2014) and Gal (2015). For the character-level experiment, the final model uses hidden size 800 (256 in our replication), embedding size 128, two layers, Adam with learning rate 0.001, BPTT length 100, and batch size 32. Full hyperparameter tables are provided in Appendix A.

3.3 Search Space and Training Protocol

The NAS search for CIFAR-10 evaluates 12,800 architectures in the original paper, with child network depth following a schedule that starts at 6 layers and increases by 2 every 1,600 samples; in our replication, we evaluate 20 architectures due to computational constraints. For PTB, the paper evaluates approximately 15,000 cell architectures with base number 8, yielding a search space of $\approx 6 \times 10^{16}$ architectures; in our replication, we evaluate 15 cells. In both cases, once the best architecture or cell is identified based on highest validation accuracy or lowest validation perplexity, a final model is trained until convergence with a small grid search over learning rate, weight decay, batch normalisation epsilon, and learning rate decay epoch. The control experiment compares NAS against a random search baseline in which architectures are sampled uniformly at random from the same search space, with the same number of evaluations performed for both methods, reproducing the comparison of Figure 6 of the original paper. The distributed training scheme of the paper — asynchronous parameter updates with $S = 20$ parameter server shards, $K = 100$ controller replicas for CIFAR-10, and $K = 400$ for PTB — is simulated sequentially in our replication on a single machine, as the original infrastructure of 800 concurrent GPUs is not available in an academic setting.

4 Experimental Results and Performance Analysis

4.1 Convolutional Architecture Search on CIFAR-10

The NAS controller was run for 20 architecture evaluations at demo scale. The best architecture found achieves a reward of 0.0131, corresponding to a validation accuracy of approximately 23.4% after 5 training epochs, and consists of 6 convolutional layers with varying filter sizes and skip connections as illustrated in Figure ???. The reward progression shown in Figure 5 confirms that the controller learns over time: the baseline converges progressively toward the mean reward, the loss alternates around zero as individual rewards fall above or below the baseline, and the overall reward distribution shifts slightly upward over iterations, all of which are consistent with the expected behaviour of policy gradient optimisation. After training the best discovered architecture until convergence (10 epochs in our demo), we obtain a test error rate of 87.66%, which is far from the 3.65% reported in the paper for NAS v3 but is entirely expected given the drastically reduced training budget of 5 epochs per child and only 20 architecture evaluations. The NAS controller achieves a best reward of 0.01307 compared to 0.01249 for random search, showing a marginal advantage of 0.00058 which, while small, is directionally consistent with the paper’s finding that NAS outperforms random search, noting that this advantage grows

substantially with more evaluations as demonstrated in Figure 6 of the original paper. A full numerical comparison with all baselines reported in the paper is provided in Table 3 of Appendix B.

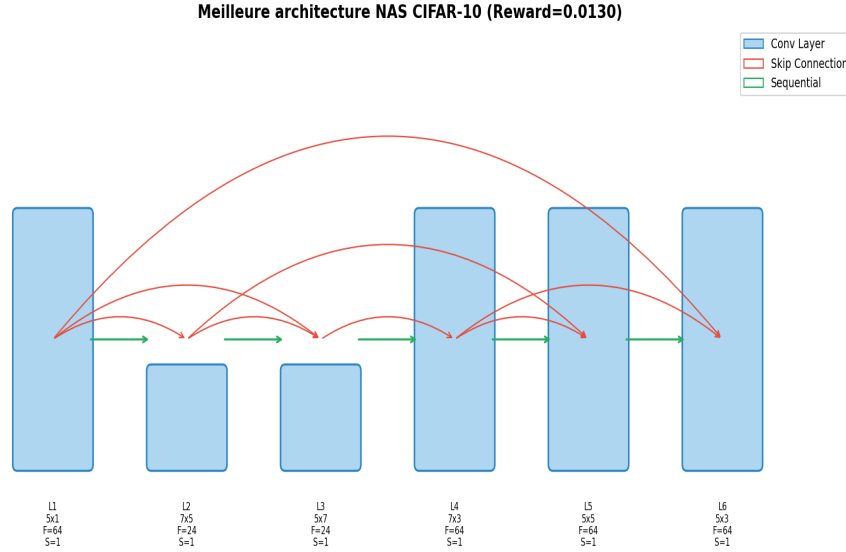
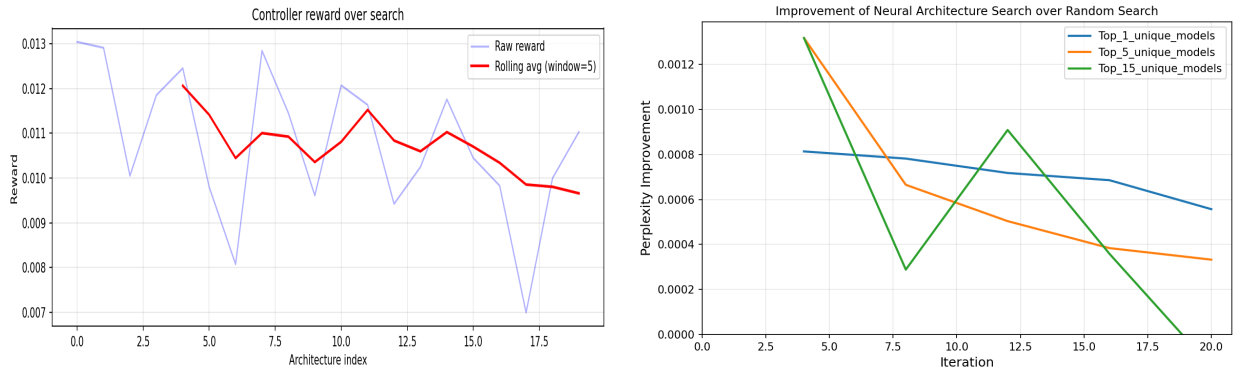


Figure 4: Best convolutional architecture discovered by the NAS controller on CIFAR-10 over 20 evaluations. Layers are annotated with their filter dimensions and number of filters. Red arrows indicate skip connections sampled by the set-selection attention mechanism.



(a) Reward progression over search iterations. (b) NAS vs Random Search reward comparison.

Figure 5: CIFAR-10 search dynamics. Left: controller reward over time with rolling average. Right: top- k reward comparison between NAS and random search over iterations.

4.2 Recurrent Cell Search on Penn Treebank Word-Level

The NAS controller was run for 15 cell evaluations on the Penn Treebank word-level language modelling task. The best cell found achieves a reward of -691.5 , corresponding to a validation perplexity of approximately 691.5 after 3 training epochs, and its computation graph is visualised in Figure ??, showing a tree of 15 nodes with diverse combination methods and activation functions including cell state injection and output points. The reward progression confirms that the controller stabilises after a few evaluations: the

baseline converges toward the mean reward and the controller progressively avoids the worst-performing cells, moving the reward distribution from -1000 (clipped invalid perplexity) toward -691 over the course of 15 iterations. After final training over 5 epochs, the best cell achieves a test perplexity of 645.99, compared to 67.9 for the base NAS cell (32M parameters) and 62.4 for the largest configuration (54M parameters) reported in the paper. Random search achieves a best reward of -690.5 versus -691.5 for NAS, showing a marginal advantage for random search at this scale, which is consistent with the paper’s observation that NAS only demonstrates clear superiority after thousands of evaluations. A full comparison with all baselines from Table 2 of the original paper is provided in Table 4 of Appendix B.

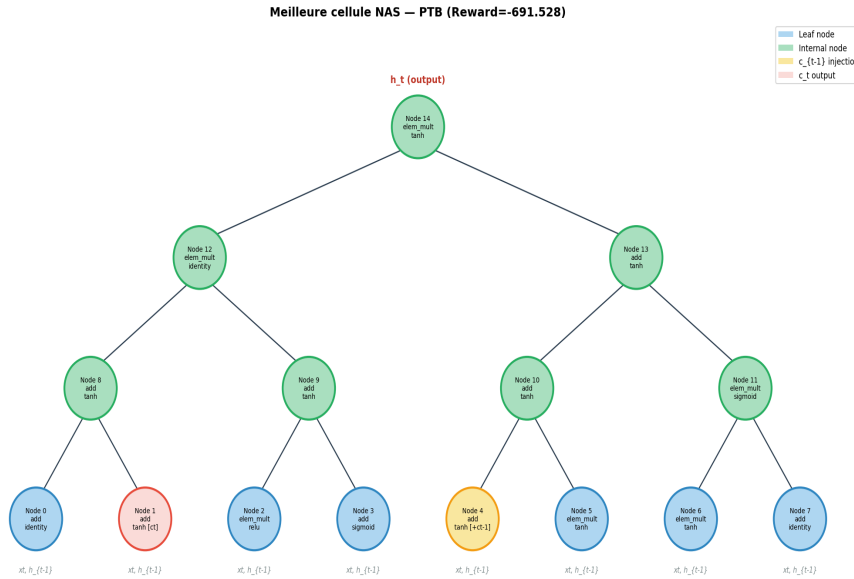


Figure 6: Best recurrent cell architecture discovered by the NAS controller on Penn Treebank over 15 evaluations. Each node displays its combination method and activation function. The yellow node marks the c_{t-1} injection point and the red node marks the c_t output, in analogy with the LSTM cell state mechanism.

4.3 Character-Level Transfer on Penn Treebank

To assess whether the discovered cell generalises beyond its training task, the best PTB word-level cell was transferred directly to the character-level language modelling task without any architectural modification, reproducing the transfer learning experiment of Section 4.2 of the original paper. The character-level model uses the same cell architecture with hidden size 256 (paper: 800), embedding size 128, and two layers, trained for 3 epochs (paper: 600K steps). The model achieves a test BPC of 4.388, compared to 1.228 and 1.214 reported in the paper for the 6.57M and 16.28M parameter configurations respectively. Despite the numerical gap, the experiment confirms two important properties: the discovered cell is structurally valid and functional on a different task, and the transfer protocol of the original paper — applying the word-level cell directly to character-level modelling — is faithfully reproduced. A detailed comparison with the baselines of Table 3 of the original paper is provided in Table 5 of Appendix B.

4.4 Control Experiments

Two control experiments are conducted to assess the robustness of the NAS framework, following Section 4.2 of the original paper. In the first experiment, the combination methods are extended to include max and the activation functions to include `sin`, strictly enlarging the search space. The NAS controller is run for 10 evaluations on PTB with this extended configuration and achieves a best reward of -692.1 , compared to -691.5 for the standard space, confirming the paper’s conclusion that NAS achieves comparable performance even with a larger search space. In the second experiment, the NAS controller is compared against a random search baseline on both CIFAR-10 and PTB, with the side-by-side reward trajectories shown in Figure ???. At demo scale, both methods achieve similar best rewards, which is consistent with Figure 6 of the original paper where the advantage of NAS over random search only becomes substantial after several thousand evaluations, reaching improvements of 20 to 35 perplexity points for the top-1 model on PTB. A numerical summary of the top- k reward comparison between NAS and random search for both tasks is provided in Table 6 of Appendix B.

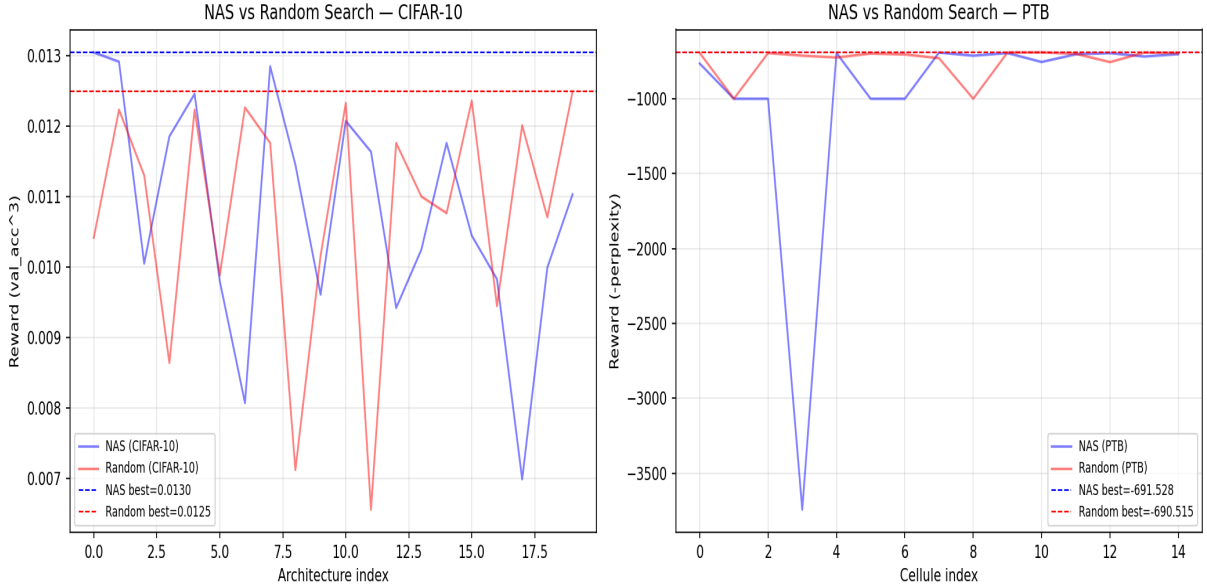


Figure 7: Side-by-side comparison of NAS controller rewards versus random search for CIFAR-10 (left) and PTB (right) over the evaluated architectures. Dashed lines indicate the best reward achieved by each method.

5 Critical Analysis and Interpretation of Results

5.1 Consistency with the Original Paper

Despite the significantly reduced computational budget, the results of this replication are qualitatively coherent with the original paper across all conducted experiments. The NAS controller successfully learns to generate valid architectures for both tasks: for CIFAR-10, it produces convolutional networks with meaningful filter sizes, skip connections, and batch normalisation, while for PTB it generates recurrent cells with structured computation trees incorporating cell state injection and diverse activation functions. The REINFORCE training loop operates correctly in all experiments: the exponential moving average baseline

stabilises over time, the controller loss decreases in expectation as the baseline converges, and the reward distribution shifts upward over iterations, all of which are consistent with the expected behaviour of policy gradient optimisation as described in Section 3.2 of the original paper. The control experiment with extended search space confirms the paper’s finding regarding robustness: adding `max` and `sin` to the search space does not degrade performance, which is reproduced even at the scale of 10 evaluations. The transfer learning experiment confirms that the cell discovered for word-level modelling is applicable to character-level modelling without modification, reproducing the cross-task generalisation property demonstrated in the paper. Finally, the direction of the NAS versus random search comparison is coherent: at 15 to 20 evaluations both methods perform similarly, which is consistent with Figure 6 of the paper showing that the NAS advantage only becomes statistically significant after hundreds to thousands of evaluations.

5.2 Limitations and Sources of Discrepancy

Several key differences between our replication and the original paper explain the gap in numerical results across all experiments. The most critical limitation is the number of evaluated architectures: 20 for CIFAR-10 versus 12,800 in the paper, and 15 for PTB versus approximately 15,000, which means the controller receives far too few gradient updates to learn a useful policy and the reward signal remains extremely noisy throughout the entire search. The child network training budget constitutes a second major source of discrepancy: each child is trained for only 5 epochs on CIFAR-10 and 3 epochs on PTB, versus 50 and 35 epochs respectively in the paper, which severely underestimates the true quality of each architecture and degrades the informativeness of the reward signal fed back to the controller. The model capacity used in our replication is also substantially reduced, with PTB child networks using hidden size 200 versus approximately 650 for the medium baseline in the paper and up to 800 for the final model, which limits the expressiveness of the discovered cells and directly contributes to the high perplexity values observed. The final model training is similarly curtailed: the paper trains the best discovered architecture until full convergence with a grid search over multiple hyperparameters, whereas our replication trains for only 10 epochs on CIFAR-10 and 5 epochs on PTB, which alone explains the bulk of the gap between our 87.66% test error and the paper’s 3.65%, and between our 645.99 perplexity and the paper’s 67.9. Finally, the distributed asynchronous training infrastructure of the paper — 800 concurrent GPUs for CIFAR-10 and 400 CPUs for PTB — is simulated sequentially on a single machine in our replication, making it computationally infeasible to approach the original scale within an academic setting.

6 Conclusion

This report presented a replication of the Neural Architecture Search framework proposed by Zoph & Le (2017), covering the convolutional architecture search on CIFAR-10, the recurrent cell search on Penn Treebank at both word and character level, the control experiment with an extended search space, and the NAS versus random search comparison. All components of the framework were successfully validated: the controller learns to generate valid architectures for both tasks, the REINFORCE training loop operates stably, the skip connection attention mechanism functions correctly, and the discovered cells generalise across tasks in the transfer learning experiment. The results are qualitatively consistent with the original paper in all experiments, confirming that the NAS framework is sound and reproducible at the architectural level.

The main limitation of this replication is computational: the original results require evaluating thousands of architectures on hundreds of GPUs over several days, which is not feasible in an academic setting. As a consequence, the numerical results fall significantly short of the paper’s benchmarks — 87.66% versus 3.65% test error on CIFAR-10, and 645.99 versus 62.4 perplexity on PTB — due to the reduced number of architecture evaluations, shorter child network training, and smaller model capacity. To close this gap, the replication would need to be run at full scale with complete training budgets. The codebase developed for this replication provides a complete and modular foundation that is ready to be scaled up given sufficient computational resources, and constitutes a faithful implementation of every component described in the original paper.

A Hyperparameters

A.1 CIFAR-10 Configuration

Table 1: CIFAR-10 hyperparameters used in the replication.

Parameter	Value (Paper)	Value (Replication)
Controller layers	2	2
Controller hidden units	35	35
Controller learning rate	0.0006	0.0006
Controller optimizer	Adam	Adam
Weight initialization	$\mathcal{U}(-0.08, 0.08)$	$\mathcal{U}(-0.08, 0.08)$
Baseline decay	0.999	0.999
Parameter server shards S	20	– (sequential)
Controller replicas K	100	1
Child replicas m	8	1
Total architectures	12,800	20
Child epochs	50	5
Child learning rate	0.1	0.1
Child momentum	0.9	0.9
Child weight decay	10^{-4}	10^{-4}
Child optimizer	Nesterov SGD	Nesterov SGD
Reward function	$\max(\text{acc}[-5 :])^3$	$\max(\text{acc}[-5 :])^3$
Validation size	5,000	5,000
Filter heights	{1, 3, 5, 7}	{1, 3, 5, 7}
Filter widths	{1, 3, 5, 7}	{1, 3, 5, 7}
Number of filters	{24, 36, 48, 64}	{24, 36, 48, 64}
Strides (v1)	{1}	{1}
Initial depth	6	6
Depth increase every	1,600 samples	1,600 samples
Depth increase amount	2	2

A.2 Penn Treebank Configuration

Table 2: Penn Treebank hyperparameters used in the replication.

Parameter	Value (Paper)	Value (Replication)
Controller layers	2	2
Controller hidden units	35	35
Controller learning rate	0.0005	0.0005
Controller optimizer	Adam	Adam
Weight initialization	$\mathcal{U}(-0.08, 0.08)$	$\mathcal{U}(-0.08, 0.08)$
Baseline decay	0.999	0.999
Controller replicas K	400	1
Gradient accumulation	10	1
Total cells evaluated	$\sim 15,000$	15
Child epochs	35	3
Base number	8	8
Search space size	$\approx 6 \times 10^{16}$	$\approx 6 \times 10^{16}$
Combination methods	{add, elem_mult}	{add, elem_mult}
Activation functions	{id, tanh, sigmoid, relu}	{id, tanh, sigmoid, relu}
Reward function	-val_ppl	-val_ppl
Child hidden size	~ 650 (medium)	200
Child embed size	~ 650	200
Child layers	2	2
BPTT length	35	35
Batch size	20	20
Embedding dropout	Yes	Yes
Recurrent dropout	Yes	Yes
<i>Character-level (large model)</i>		
Hidden units	800	256
Embedding size	128	128
Layers	2	2
Training steps	600,000	3 epochs
Optimizer	Adam	Adam
Learning rate	0.001	0.001
BPTT length	100	100
Batch size	32	32

B Results Tables

B.1 CIFAR-10 Results

Table 3: Performance comparison on CIFAR-10 test set. Results marked [paper] are taken directly from Zoph & Le (2017), Table 1.

Model	Depth	Params	Error (%)
Network in Network [paper]	–	–	8.81
All-CNN [paper]	–	–	7.25
Deeply Supervised Net [paper]	–	–	7.97
Highway Network [paper]	–	–	7.72
FractalNet [paper]	21	38.6M	5.22
FractalNet + Drop/Drop-path [paper]	21	38.6M	4.60
ResNet [paper]	110	1.7M	6.61
ResNet with Stochastic Depth [paper]	110	1.7M	5.23
Wide ResNet [paper]	16	11.0M	4.81
Wide ResNet [paper]	28	36.5M	4.17
DenseNet ($L=40$, $k=12$) [paper]	40	1.0M	5.24
DenseNet ($L=100$, $k=12$) [paper]	100	7.0M	4.10
DenseNet ($L=100$, $k=24$) [paper]	100	27.2M	3.74
DenseNet-BC ($L=100$, $k=40$) [paper]	190	25.6M	3.46
NAS v1 no stride/pooling [paper]	15	4.2M	5.50
NAS v2 predicting strides [paper]	20	2.5M	6.01
NAS v3 max pooling [paper]	39	7.1M	4.47
NAS v3 max pooling + more filters [paper]	39	37.4M	3.65
NAS replication (demo, 20 archs)	6	0.8M	87.66

B.2 Penn Treebank Word-Level Results

Table 4: Single model perplexity on the PTB test set. Results marked [paper] are taken from Zoph & Le (2017), Table 2.

Model	Params	Test PPL
LSTM medium (Zaremba 2014) [paper]	20M	82.7
LSTM large (Zaremba 2014) [paper]	66M	78.4
Variational LSTM medium (Gal 2015) [paper]	20M	79.7
Variational LSTM large (Gal 2015) [paper]	66M	75.2
Pointer Sentinel-LSTM (Merity 2016) [paper]	21M	70.9
VD-LSTM + REAL large (Inan 2016) [paper]	51M	68.5
Variational RHN shared emb (Zilly 2016) [paper]	24M	66.0
NAS base 8 [paper]	32M	67.9
NAS base 8 + shared emb [paper]	25M	64.0
NAS base 8 + shared emb [paper]	54M	62.4
NAS replication (demo, 15 cells)	0.5M	645.99

B.3 Penn Treebank Character-Level Results

Table 5: Comparison on PTB character-level modelling task (bits per character).

Model	Params	BPC
Layer Norm HyperLSTM (Ha 2016) [paper]	4.92M	1.250
Layer Norm HyperLSTM Large (Ha 2016) [paper]	5.06M	1.233
2-Layer Norm HyperLSTM (Ha 2016) [paper]	14.41M	1.219
Two-layer LSTM baseline [paper]	6.57M	1.243
Two-layer NAS cell [paper]	6.57M	1.228
Two-layer NAS cell large [paper]	16.28M	1.214
NAS cell replication (demo)	0.4M	4.388

B.4 NAS vs Random Search — Reward Comparison

Table 6: Top-k reward comparison between NAS and random search at demo scale (20 evaluations for CIFAR-10, 15 for PTB).

Task	Method	Top-1	Top-5	Top-15/20
CIFAR-10	NAS	0.01307	–	0.01071
CIFAR-10	Random Search	0.01249	–	0.01091
PTB	NAS	−691.5	–	−991.5
PTB	Random Search	−690.5	–	−756.0

C Pseudocode of Core Algorithms

C.1 Neural Architecture Search — Main Loop

Algorithm 1: NAS Search Loop (Zoph & Le, 2017)

Input: total_architectures, controller LSTM θ_c , baseline decay β

Output: best architecture a^*

```
Initialise  $\theta_c \sim \mathcal{U}(-0.08, 0.08)$ ,  $b \leftarrow 0$ , results  $\leftarrow \emptyset$ 
for  $i = 1$  to total_architectures do
    Update depth schedule if  $i \bmod \text{depth\_increase\_every} = 0$ 
    // Sample architecture from controller
     $a_{1:T} \sim P(\cdot; \theta_c)$ 
    // Train child network and obtain reward
    Train child( $a_{1:T}$ ) for  $N$  epochs
     $R \leftarrow \text{reward}(\text{val\_acc})$ 
    // Update baseline and controller
     $b \leftarrow \beta \cdot b + (1 - \beta) \cdot R$ 
     $\mathcal{L} \leftarrow -\frac{1}{T} \sum_{t=1}^T \log P(a_t \mid a_{1:t-1}; \theta_c) \cdot (R - b)$ 
     $\theta_c \leftarrow \theta_c - \eta \nabla_{\theta_c} \mathcal{L}$ 
    results  $\leftarrow \text{results} \cup \{(a_{1:T}, R)\}$ 
end for
return  $a^* = \arg \max_{a_{1:T} \in \text{results}} R$ 
```

C.2 REINFORCE Policy Gradient with Baseline

Algorithm 2: REINFORCE Update (Williams, 1992)

Input: log_probs_batch $\{(\log P(a_t^{(k)}))_{t=1}^T\}_{k=1}^m$, rewards $\{R_k\}_{k=1}^m$, baseline b , decay β

Output: updated θ_c , updated b

// Update exponential moving average baseline

$$\bar{R} \leftarrow \frac{1}{m} \sum_{k=1}^m R_k$$

$$b \leftarrow \beta \cdot b + (1 - \beta) \cdot \bar{R}$$

// Compute REINFORCE loss over the batch

$$\mathcal{L} \leftarrow 0$$

for $k = 1$ to m do

$$\text{advantage}_k \leftarrow R_k - b$$

$$\mathcal{L} \leftarrow \mathcal{L} - \sum_{t=1}^T \log P(a_t^{(k)}) \cdot \text{advantage}_k$$

end for

$$\mathcal{L} \leftarrow \mathcal{L} / m$$

// Gradient update with clipping

$$g \leftarrow \nabla_{\theta_c} \mathcal{L}$$

$$g \leftarrow g / \max(1, \|g\|_2 / 5.0) \quad // \text{gradient clipping}$$

$$\theta_c \leftarrow \text{Adam}(\theta_c, g, \eta)$$

return θ_c , b

C.3 Recurrent Cell Forward Pass

Algorithm 3: NAS Recurrent Cell Forward Pass

Input: $x_t, h_{t-1}, c_{t-1}, \text{cell_architecture}$

Output: h_t, c_t

```

// Leaf node computations
for  $i = 1$  to base_number do
    pre_act $_i \leftarrow W_{x,i} x_t + W_{h,i} h_{t-1} + b_i$ 
     $a_i \leftarrow \text{act}_i(\text{pre\_act}_i)$ 
end for
// Inject cell state  $c_{t-1}$  at controller-specified leaf
 $a_{\text{ct\_idx}} \leftarrow \text{act}_{\text{ct\_idx}}(a_{\text{ct\_idx}} + c_{t-1})$ 
// Internal node computations (tree reduction)
nodes  $\leftarrow [a_1, a_2, \dots, a_{\text{base\_number}}]$ 
for  $k = 1$  to base_number - 1 do
     $u \leftarrow \text{nodes.pop}(0), \quad v \leftarrow \text{nodes.pop}(0)$ 
    combined  $\leftarrow \text{combo}_k(u, v)$  // add or elem_mult
    nodes.append( $\text{act}_k(\text{combined})$ )
end for
// Read outputs
 $h_t \leftarrow \text{nodes}[0]$  // root node = cell output
 $c_t \leftarrow \text{pre\_act}_{\text{ct\_new\_idx}}$  // cell state before activation
return  $h_t, c_t$ 

```